EC-220 Computer System Architecture

Dr. Ahsan Shahzad

Department of Computer and Software Engineering (DC&SE)

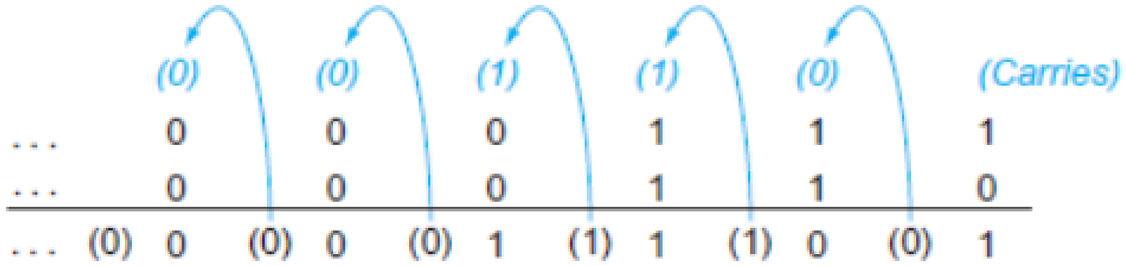
College of Electrical and Mechanical Engineering (CEME)

National University of Science and Technology (NUST)

Week # 7a_Corona Break

Arithmetic Operations– Addition and Subtraction

- **Addition:** Bit by bit from right to left, with carries passed to the next digit to the left.
- **Subtraction:** By using addition operation; negate the appropriate operand and then add



$$
\begin{array}{ll}
 & 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0111_{two} = 7_{ten} \\
- & 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0110_{two} = 6_{ten} \\
\hline
= & 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0001_{two} = 1_{ten}
\end{array}
$$

or via addition using the two's complement representation of −6:

$$
\begin{array}{ll}
 & 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0111_{two} = 7_{ten} \\
+ & 1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1010_{two} = -6_{ten} \\
\hline
= & 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0001_{two} = 1_{ten}
\end{array}
$$

Overflow

- No overflow when adding a +ve and a -ve number e.g. -10 + 5 = -5
- No overflow when signs are the same for subtraction e.g. 10 – 5 = 5

- **Signed Overflow** occurs when the value affects the sign:
  - Addition
    - overflow when adding two +ves yields a –ve
    - Or, adding two -ves gives a +ve
  - Subtraction
    - subtract a -ve from a +ve and get a –ve
    - or, subtract a +ve from a -ve and get a +ve
- **Unsigned Overflow:** unsigned nos. normally used to represent memory addresses where overflows are ignored

MIPS Assembly – Addition and Subtraction

- **add, addi, sub** will cause exceptions on overflow
- **addu, addiu, subu** do not cause exceptions on overflow
- Don't always want to detect overflow
  - new MIPS instructions: addu, addiu, subu

note: addiu *still sign-extends!*

note: sltu, sltiu *for unsigned comparisons*

Effects of Overflow

- MIPS detects overflow with an **exception**, also called an **interrupt** on many computers.

- MIPS throws an exception (interrupt) upon overflow
  - Asynchronous and unscheduled procedure call
  - Jump to predefined address (e.g. set by the OS)
  - Recoverable or non-recoverable
  - EPC (*exception program counter*) holds (PC) address of instruction that trigger exception
  - Mfc0 (*move from system control*) retrieves EPC to general-purpose Reg. to jump back using Jr intr., if recovery possible

Checking for Overflow

```
#addition...
addu    $t0, $t1, $t2                 # caution, don't trap!
xor     $t3, $t1, $t2                 # if signs differ $t3 < 0
slt     $t3, $t3, $zero               # $t3 =1, if sign differs
bne     $t3, $zero, No_overflow       #signs differ
xor     $t3,$t0,$t1                   # signs match, check result
slt     $t3,$t3,$zero                 #sum sign different
bne     $t3,$zero, Overflow           # signs differ  b/w result and operand - > overflow
```

Checking for Overflow

- Doesn't make much sense for unsigned
- For signed a challenge:
  - Some processors have a special instruction
  - Others, like MIPS, need assembly to do it
  - A check must always be present!
- Exception handling needs context switch
  - Save all registers
  - But EPC needs one register
  - MIPS reserves two registers, $k0 and $k1 for the OS
  - User-level software abstains from using them

Review: One bit Adder

- Takes three input bits and generates two output bits
- Multiple bits can be cascaded

Half-Adder

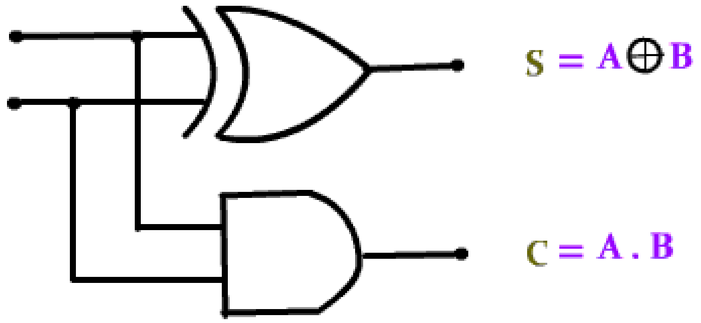
A	B	S	C

INPUT

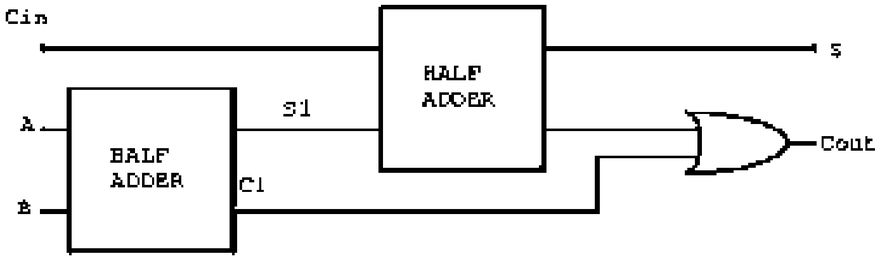
A

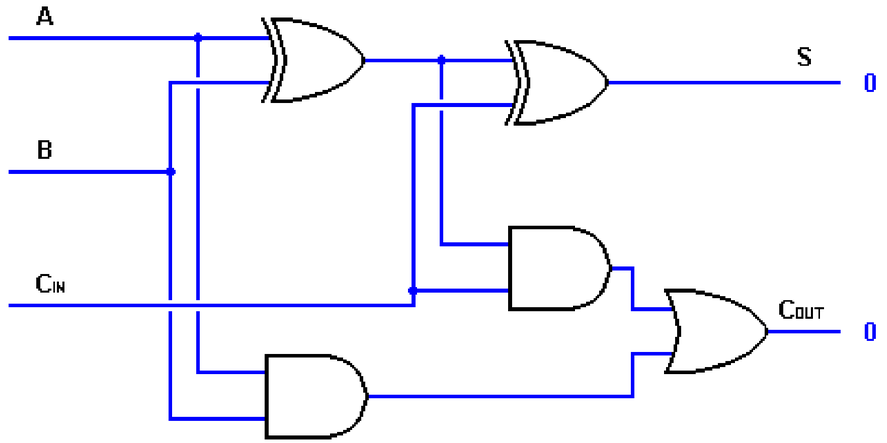
B

OUTPUT



C_1	X_1	Y_1	Z_1	C_2
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1





Problem: ripple carry adder is slow

- Is there more than one way to do addition?
  - two extremes: ripple carry and sum-of-products

Can you see the ripple? How could you get rid of it?

$c_1 = b_0c_0 + a_0c_0 + a_0b_0$

$c_2 = b_1c_1 + a_1c_1 + a_1b_1$

$c_3 = b_2c_2 + a_2c_2 + a_2b_2$

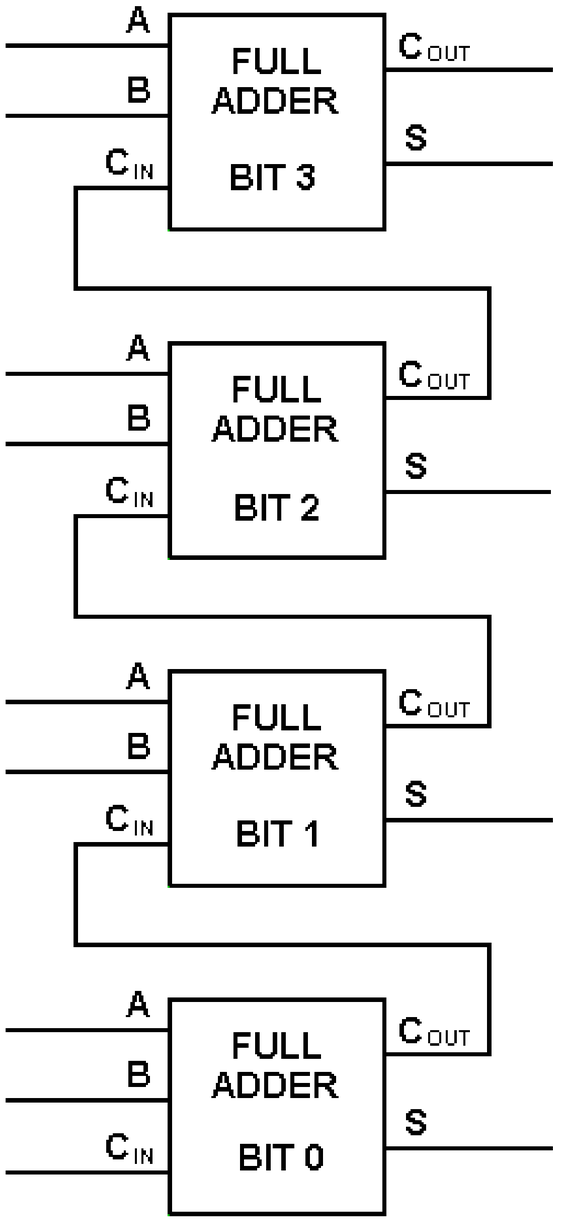
$c_4 = b_3c_3 + a_3c_3 + a_3b_3$

$c_2 =$

$c_3 =$

$c_4 =$

Not feasible! Why?



Carry-look-ahead adder

An approach in-between our two extremes

- Motivation:
  - If we didn't know the value of carry-in, what could we do?
  - When would we always generate a carry? $g_i = a_i\, b_i$
  - When would we propagate the carry? $p_i = a_i + b_i$
- Did we get rid of the ripple?

$$c_1 = g_0 + p_0c_0$$

$$c_2 = g_1 + p_1c_1 \qquad c_2 = g_1+p_1g_0+p_1p_0c_0$$

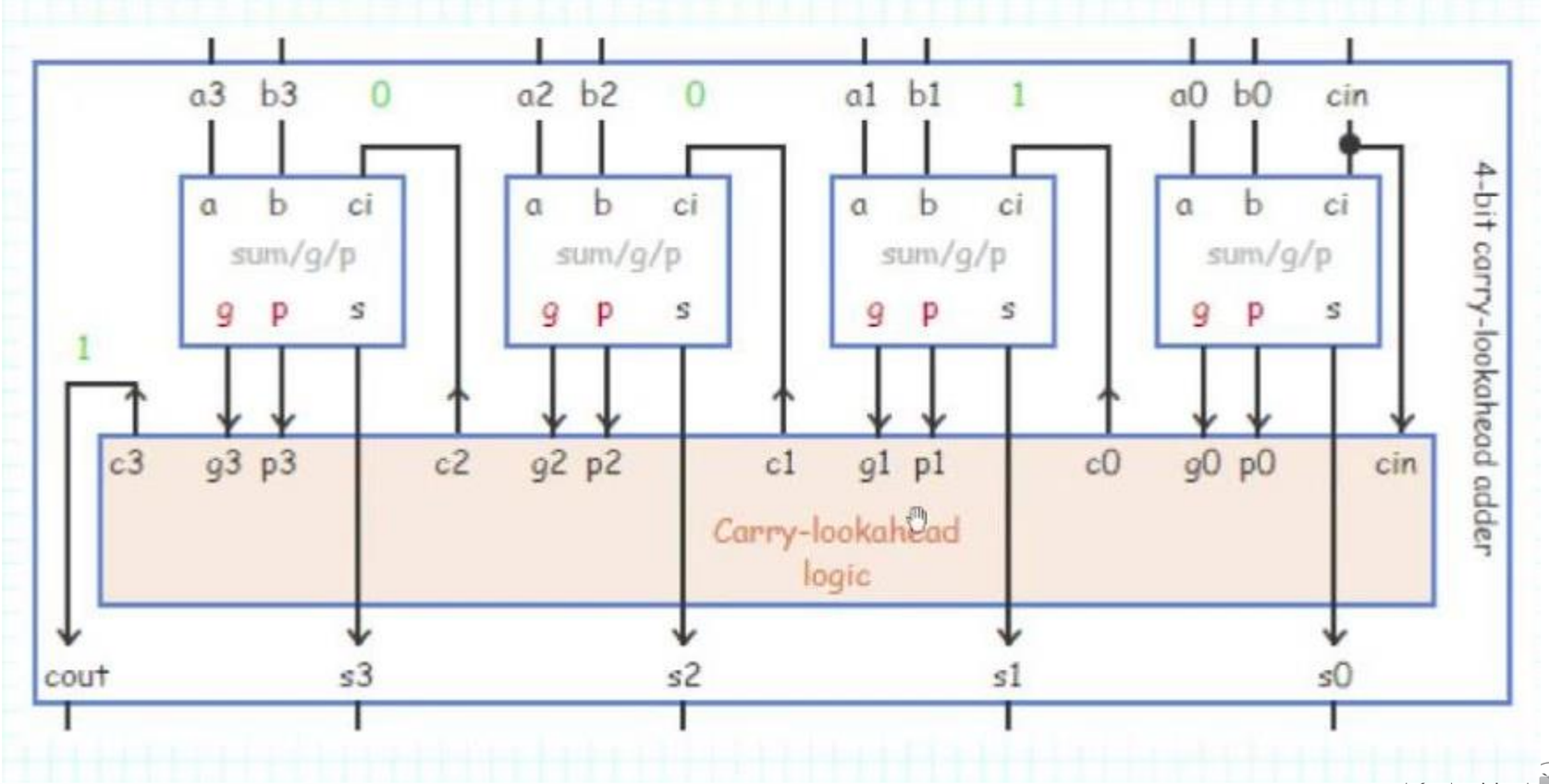
$$c_3 = g_2 + p_2c_2 \qquad c_3 = g_2+p_2g_1+p_2p_1g_0+p_2p_1p_0c_0$$

$$c_4 = g_3 + p_3c_3 \qquad c_4 = g_3+p_3g_2+p_3p_2g_1+p_3p_2p_1g_0+p_3p_2p_1p_0c_0$$

Feasible! Why?

A 4-bit carry look-ahead adder

- Generate g and p term for each bit
- Use g's, p's and carry in to generate all C's
- Also use them to generate block G and P



Use principle to build bigger adders

- CLA principle can be used recursively
- A 16 bit adder uses four 4-bit adders
- It takes block g and p terms and cin to generate block carry bits out
- Block carries are used to generate bit carries
  - could use ripple carry of 4-bit CLA adders
  - Better: use the CLA principle again!